Computers & Security 173 (2027) 105209

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Same publisher, changed authority: Release-bound semantic differencing of PyPI attestations

Qingshui Xue *, Pandong Xue, Zhimin Wang, Haifeng Ma

Faculty of Intelligent Technology, Shanghai Institute of Technology, Shanghai 201418, China

ARTICLE INFO

Keywords:
Software supply chain security
Package provenance
PyPI
GitHub Actions
Trusted Publishing
Semantic differencing

ABSTRACT

Package attestations bind a release to a workload identity and source commit, but an unchanged publisher identity can conceal a change in the release-bound workflow's authority. This creates a longitudinal blind spot between identity-transition measurement and snapshot scanning. AttestDiff addresses it by defining the same-identity attested workflow pair as a security-review unit. It reconstructs exact workflows at attested commits, projects each onto the transitive publishing slice, and applies guarded directional rules that retain authority expansions, restrictions, incomplete-context signals, and unknowns with old/new evidence. A frozen study of 350 popular PyPI projects reconstructed 862 workflow snapshots and formed 720 same-identity release pairs. AttestDiff retained 59 pairs (8.2%; project-clustered 95% CI: 6.0%–10.6%), compared with 236 (32.8%) from an independent Zizmor temporal baseline, 129 (17.9%) from an internal rule-output delta, and 349 (48.5%) raw workflow changes. Two external reviewers without prior project involvement independently assessed all 59 candidates and a seeded 59/290 non-alert sample. First-round four-class agreement was 87.3% (Cohen's $\kappa = 0.743$). After clarification and joint adjudication, 33/59 candidates (55.9%) warranted security review, 21 were benign maintenance, and five required unavailable public context; all 59 sampled controls were judged benign. The candidate-minus-control difference was 55.9 percentage points (project-clustered 95% CI: 41.8–69.7). These results support reconstruction feasibility, queue distinction, and enrichment for codebook-defined review-worthiness; they do not establish vulnerability prevalence or complete recall.

1. Introduction

Modern package publication is moving from reusable registry credentials toward short-lived workload identities and verifiable provenance. PyPI Trusted Publishing, PEP 740 attestations, Sigstore, SLSA, and GitHub artifact attestations make the origin and build path of a release more observable (Ingram, 2023; Woodruff et al., 2024; Ingram, 2024; Newman et al., 2022; Open Source Security Foundation, 2023; GitHub, 2024). This shift addresses a real weakness: long-lived secrets and weak release authentication remain recurring supply chain concerns (Schorlemmer et al., 2024b; Kalu et al., 2025). Yet provenance answers a deliberately bounded question. PyPI states that an attestation records where a distribution came from; it does not establish that the source or publishing workflow is benign (Python Package Index, 2026b,a).

That boundary creates a longitudinal review problem. Two consecutive release files can name the same OpenID Connect issuer, repository, workflow path, and publisher environment while the referenced workflow changes its triggers, token permissions, OIDC authority, runner boundary, dependency pinning, or artifact handoff. Identity monitoring sees continuity. A snapshot scanner can repeatedly report a persistent state. A repository-history diff sees many unrelated edits. None of these units directly asks: *what publishing-relevant authority changed between the exact attested workflow snapshots of two releases under the same identity?*

The distinction matters because GitHub Actions is both an automation platform and a supply-chain execution boundary. Prior studies have documented command-injection paths, poisoned caches, mutable action dependencies, and privileged workflow configurations (Muralee et al., 2023; Gu et al., 2024, 2023a; Chaiwut and Nikiforakis, 2025). However, ordinary workflow maintenance is frequent and usually small (Mazrae et al., 2026); equating every textual change with a security regression would create an unhelpful queue. Conversely, reducing a comparison to a binary "changed/not changed" state loses whether authority broadened, narrowed, or became unresolvable.

We present AttestDiff, a release-bound semantic differencer for PyPI attestations. It uses the attestation as an anchor to retrieve the workflow bytes at the named commit, gates comparisons on an unchanged publisher identity, projects both snapshots onto the publishing path, and records old/new evidence for each directional semantic change.

* Corresponding author.
E-mail addresses: xue-qsh@sit.edu.cn (Q. Xue), 256092459@mail.sit.edu.cn (P. Xue), wangzhimin@sit.edu.cn (Z. Wang), mahf@sit.edu.cn (H. Ma).

https://doi.org/10.1016/j.cose.2026.105209
Received 29 July 2026; Received in revised form 29 September 2026; Accepted 13 October 2026
Available online 31 October 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

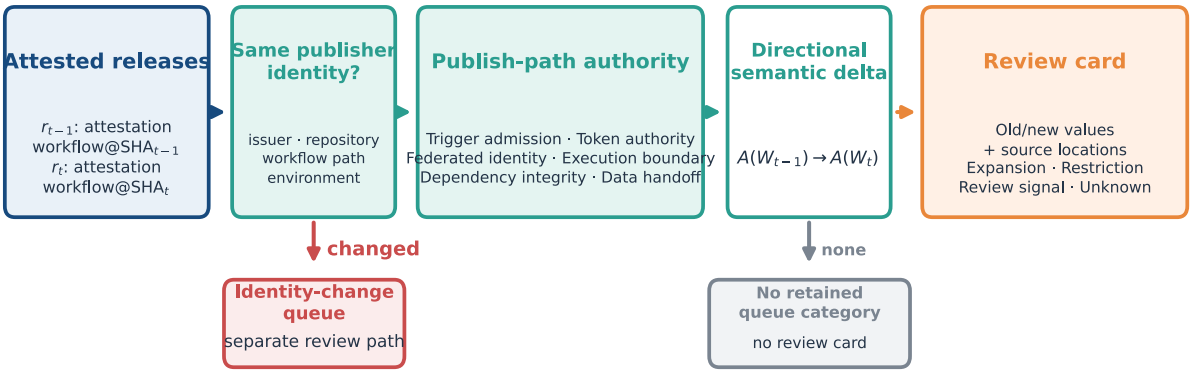


Fig. 1. AttestDiff release-bound semantic-differencing workflow.
Note. Identity transitions leave through a separate branch. Same-identity snapshots enter publishing-path projection and directional comparison; only pairs containing a frozen candidate category yield review cards, while other pairs take the gray no-card exit. A review card retains evidence and unresolved context rather than assigning a vulnerability label.

Fig. 1 shows the five-step analysis and its explicit off-ramps for identity changes and same-identity pairs without a candidate category. The output is a review card, not a vulnerability label.

We study four research questions:

- **RQ1: Reconstruction.** Can public PyPI provenance reconstruct exact release-bound workflow snapshots at useful scale?
- **RQ2: Semantic change.** Which observable authority semantics change while publisher identity remains stable?
- **RQ3: Selectivity and distinction.** How does the candidate queue compare with structural, single-snapshot, internal temporal-rule, and independent-scanner baselines?
- **RQ4: Robustness and reviewability.** Do the findings survive adjacency and within-release file sensitivities, and does the candidate queue concentrate transitions that two stratum-blinded external reviewers judge review-worthy under the frozen codebook?

This work makes four contributions:

1. It defines the *same-identity attested workflow pair* as a security-review unit distinct from a release-authority transition, a generic repository edit, and a workflow snapshot.
2. It introduces a guarded semantic taxonomy and algorithm that preserve expansions, restrictions, review signals, and unknowns over the attested publishing path.
3. It reports a frozen public-data measurement with six operational baselines, including an independent Zizmor temporal baseline, together with project-clustered uncertainty, strict-adjacency and cross-file sensitivities, and a concrete review-card exemplar.
4. It provides an anonymous supplementary reproducibility package with the implementation and environment, frozen protocols, dated sampling frame, content-hashed evidence, a de-identified 720-pair ledger, the 20-case conformance suite, independent-scanner outputs, analysis and figure scripts, source-sheet hashes, and de-identified row-level judgments from the completed external audit. The audit quantifies candidate enrichment without treating review-worthiness as vulnerability prevalence.

2. Background and related work

2.1. Provenance, signing, and trusted publishing

in-toto formalized signed statements about authorized supply-chain steps and materials (Torres-Arias et al., 2019); Sigstore lowered operational barriers to identity-based signing through short-lived certificates and a transparency log (Newman et al., 2022). SLSA provides a provenance vocabulary and progressive assurance levels (Open Source Security Foundation, 2023). PEP 740 adapts attestations to Python package indexes; PyPI deployed that support in 2024, and PEP 807 is extending digital-attestation support across indexes (Woodruff et al., 2024; Ingram, 2024; Woodruff, 2025). PyPI Trusted Publishing binds uploads to configured CI identities rather than static API tokens (Ingram, 2023). GitHub's 2024 artifact-attestation release provides a parallel Actions-native provenance path (GitHub, 2024).

Recent empirical work shows that adoption and interpretation remain non-trivial. Schorlemmer et al. measured signing quantity and quality across four registries and separately studied provenance adoption factors (Schorlemmer et al., 2024b,a). Industry interviews and maintainer studies identify usability, platform support, and operational cost as central to security-feature adoption (Kalu et al., 2025; Ayala et al., 2025; Kalu et al., 2026). DiVerify addresses policy consistency across multiple software identities (Okafor et al., 2024). These works motivate evidence that is both machine-checkable and reviewable. AttestDiff does not propose another signing scheme; it uses existing attested identity and commit claims to compare the public authority exercised behind a stable identity.

2.2. GitHub actions security and evolution

GitHub Actions workflows can execute untrusted data, invoke external actions, access repository secrets, and request write or cloud-identity authority. Large-scale studies have characterized workflow usage and supply-chain weaknesses (Koishybayev et al., 2022; Gu et al., 2023a). ARGUS detects injection paths with inter-procedural taint analysis (Muralee et al., 2023); cache-focused work shows that dependency reuse can cross trust boundaries (Gu et al., 2024). Studies of JavaScript actions and mitigations quantify action-specific security practices (Delicheh et al., 2024; Delicheh and Mens, 2024). Broader ecosystem work has examined registry threats, hidden maintenance costs, and developer perceptions (Gu et al., 2023b; Ladisa et al., 2023; Valenzuela-Toledo et al., 2024; Saroar and Nayebi, 2023).

The closest recent GitHub Actions studies differ in their evidence units. *Time for Actions* longitudinally studies Marketplace actions and remote (Chaiwut and Nikiforakis, 2025). *Action Required* measures recommended practices in 338,812 repositories and studies developer barriers (Kubo et al., 2026). Fares et al. compare nine workflow scanners over a weakness taxonomy (Fares et al., 2026). Rostami Mazrae et al. analyze 3.4 million historical workflow versions, establishing that workflow evolution is common but not release-bound (Mazrae et al., 2026). GitHub continues to harden the platform, but its own guidance still treats pinning, least privilege, and OIDC configuration as distinct controls (GitHub, 2026b,c,a).

2.3. Predecessor-aware security review

Temporal comparison has a different purpose from snapshot scanning. Pinning research demonstrates security–maintenance trade-offs

Table 1
Comparison with the closest work.

Work	Primary unit	Predecessor	Same-ID semantics	Release-bound exact snapshot	Directional change	Primary output
On Good Authority (2026) (Santos-Grueiro, 2026)	Release-authority record	Y	–	Partial	Y	Cross-ecosystem release-path review
Workflow evolution (2026) (Mazrae et al., 2026)	Repository workflow version	Y	Partial	–	Partial	Maintenance taxonomy and prevalence
Action Required (2026) (Kubo et al., 2026)	Repository practice snapshot	–	–	–	–	Practice adoption and developer barriers
Scanner comparison (2026) (Fares et al., 2026)	Workflow snapshot	–	–	–	–	Weakness findings and scanner coverage
Time for Actions (2025) (Chaiwut and Nikiforakis, 2025)	Marketplace action history	Y	–	–	Partial	Longitudinal action supply-chain risk
AttestDiff (this work)	Same-identity attested workflow pair	Y	Y	Y	Y	Evidence-backed semantic review card

Note."Release-bound exact snapshot" means that workflow bytes are retrieved at the commit named by a package file's attestation. Partial means that the evidence may include the field, but the field is not the study's comparison unit.

that a single-state rule cannot capture (He et al., 2025). The nearest conceptual work, *On Good Authority*, compares predecessor release-authority records across package ecosystems and surfaces changes in publisher, repository, workflow, provenance, signing, and mediation evidence (Santos-Grueiro, 2026). It is broader in ecosystem coverage and has completed practitioner review. Its documented boundary includes same-path compromise and unchanged compromised CI. *On Good Authority* measures public release-path transitions across ecosystems; AttestDiff conditions on an unchanged four-field attested publisher identity and then compares exact release-bound workflow snapshots for internal publishing-authority changes. It cannot detect a compromise that leaves both identity and public workflow semantics unchanged (see Table 1).

3. Method

3.1. Evidence model and claim boundary

For an attested release-file record r, define the public publisher identity

$$I(r) = (issuer, repository, workflowPath, \\ publisherEnvironment). \quad (1)$$

Let $c(r)$ be the source commit named by its certificate claims and $W(r)$ the bytes retrieved from $workflowPath$ at $c(r)$. The analysis unit is a chronological predecessor pair $p = (r_{t-1}, r_t)$ for the same project. A pair enters semantic comparison only if both workflow snapshots are reconstructable and $I(r_{t-1}) = I(r_t)$. Identity changes are routed to a separate release-authority branch.

This is an observational model. We consume PyPI-served provenance and decode its certificate claims, but do not independently repeat full cryptographic verification for every file. Public YAML cannot reveal repository token defaults, branch protection, environment approval rules, organization policy, private reusable workflows, or all runtime-resolved expressions. Therefore, AttestDiff claims an *observable semantic change*, not exploitability, malice, or compromise. The publisher-environment claim in $I(r)$ is distinct from job-level `environment:` declarations observed within the publishing slice.

3.2. Threat model and security objective

The protected assets are the integrity of a published distribution and the authority exercised along its release path: repository contents, workflow tokens, OIDC identities, protected environments, dependencies, runners, and artifact handoffs. We consider an adversary who compromises an authorized maintainer or CI principal, or otherwise obtains the ability to modify a public publishing workflow before a later release while preserving the four-field publisher identity. The same observable changes can also arise from benign maintenance; intent is therefore not an input to the analysis.

PyPI's attestation service and GitHub's immutable commit-addressed source form the evidence boundary. We assume that the public endpoints return the served attestation and workflow bytes faithfully at collection time, and we hash both for later verification. Within that boundary, the security objective is to identify release-to-release changes in observable publishing authority that warrant focused review. AttestDiff is not a prevention control and does not decide whether a release is malicious.

This release-bound boundary follows the evidence that can be tied to one release: the attestation and commit-addressed workflow are independently retrievable and can be preserved with content hashes. Historical repository policy, private reusable workflows, and runner or environment state are not consistently reconstructable from that public record; counting them as observed would mix verifiable evidence with unavailable context.

Out of scope are registry or GitHub source-service compromise, private repository policy, environment protection rules, private reusable workflows, runner-image compromise, mutable action or runtime-resource changes that leave the workflow bytes unchanged, and malicious code changes outside the six semantic families. A compromise that preserves publisher identity and public workflow semantics is intentionally undetectable. The corpus contains no ground-truth compromise labels, so this study does not estimate how often such invisible compromises occur.

3.3. Exact reconstruction and publishing slice

For each selected distribution file, AttestDiff queries the PyPI Integrity API, retains the provenance response and certificate hashes, and extracts the issuer, repository, workflow URI, source/workflow SHA, publisher environment, event, and Python Package Index (2026a). It resolves the workflow path from the certificate URI, retrieves the raw file at the exact SHA, and stores the URL, status, content SHA-256, and parsed record. Provenance from a non-GitHub publisher is retained as out of scope rather than counted as a failed GitHub reconstruction.

The parser preserves YAML's `on` key and normalizes literal triggers, filters, permissions, jobs, `needs` edges, runners, job environment declarations, and `uses:` references. Publishing sinks include the PyPA publish action and command forms for `twine`, `uv`, `poetry`, `flit`, and `hatch`. The publishing slice $P(W)$ consists of all sink jobs and

Table 2
Guarded publishing-path semantic taxonomy.

Semantic family	Compared state	Expansion or restriction	Review signal	Unknown guard
Trigger admission	Events; literal branch, tag, and path filters	New event or provable filter broadening; reverse is restriction	Additional context marker when newly admitted event is `workflow_run` or `pull_request_target`	Dynamic trigger or filter expression
Token authority	Effective explicit permissions on the publishing slice	`none`→`read/write` or `read`→`write`; reverse is restriction	Changed dynamic permission expression with observable old/new source text	Unresolved effective permission value that prevents comparison
Federated identity	`id-token` permission; job environment declaration	New explicit `id-token: write`; reverse is restriction	Job environment declaration added, removed, or changed	Job environment protection semantics are private or dynamically resolved
Execution boundary	Runner labels and reusable-workflow edges	New literal self-hosted admission	New or changed local reusable workflow	Matrix- or expression-resolved runner
Dependency integrity	`uses:` references on the publishing slice	Full commit SHA replaced by a mutable reference; reverse is restriction	Newly introduced mutable dependency reference	Unavailable private dependency
Data handoff	Cache restore and artifact download on the publishing slice	Not assigned a monotone authority direction	New cache restore or artifact download	Dynamic key, name, or path

Note. Special triggers receive a generic expansion and a separate context marker. A changed dynamic expression is a review signal when its old/new source text is observable but no monotone authority direction is justified. Unknown is reserved for an unresolved effective value that prevents a reliable comparison.

their transitive job ancestors. This retains steps that can shape the released artifact or authority used by the publishing job while excluding unrelated test and documentation jobs.

3.4. Guarded semantic projection

The projection $A(P(W))$ records six semantic families (Table 2): trigger admission, token authority, federated identity, execution boundary, dependency integrity, and data handoff. Direction is assigned only when the observable states admit an ordering. For example, `none`→`write` under complete explicit permission context is an expansion, and the reverse is a restriction. New OIDC write access, a new literal trigger, provable filter broadening, a full-SHA-to-mutable-reference change, and literal self-hosted admission are also observable expansions. A newly admitted `workflow_run` or `pull_request_target` event receives both the generic event-expansion entry and an additional context-sensitive review-signal entry; the latter does not override the literal admission broadening. The two ledger entries may contribute to one retained pair and to more than one atomic or category instance. New cache restore, artifact download, newly introduced mutable dependency reference, a job environment declaration added, removed, or changed, a changed local reusable workflow, or a changed dynamic authority expression whose old and new source expressions are observable but not directionally ordered is retained as a *review signal*: its security meaning depends on context not fully represented in the pair. An unresolved effective value that prevents a reliable comparison is *unknown*.

For a pair p, the atomic ledger is

$$\Delta(p) = A(P(W(r_t))) \ominus A(P(W(r_{t-1}))). \tag{2}$$

Each delta stores its family, category, old/new values, direction, confidence, and source evidence. A pair enters the candidate queue if at least one delta has direction *expansion* or *review signal* and belongs to the frozen security-relevant category set. Restrictions and unknowns remain on a retained card when they co-occur, but do not independently imply an unsafe release.

Release-bound semantic differencing

input: ordered release records R
for each selected file $r \in R$:
  retrieve and hash PyPI provenance
  decode $I(r)$, $c(r)$, and workflow path
  retrieve and parse $W(r)$ at $c(r)$
for each adjacent reconstructable pair (r_{t-1}, r_t):
  if $I(r_{t-1}) \neq I(r_t)$: emit identity-change record
  else:
    compute $P(W_{t-1}), P(W_t)$ and guarded projections
    compute directional ledger $\Delta(p)$
    if a frozen candidate category is present:
      emit identity, old/new values, confidence, hashes, and URLs

Algorithm 1 AttestDiff pseudocode. Evidence is retained before interpretation, and identity transitions are not mixed with same-identity semantics.

4. Study design

4.1. Frozen sample and statistical units

The core protocol, taxonomy, metrics, random seed, and stop gates were frozen before the project-scale run. The post-review independent-scanner and external audit protocols were separately frozen before their respective full runs; they did not change the corpus or AttestDiff's rules. We froze a content-hashed Top-PyPI 30-day ranking snapshot retrieved on 26 July 2026 at 17:34:07 UTC (27 July 2026 at 01:34:07 China Standard Time, UTC+8); the source reported a last update of 1 July 2026. This convenience frame is not a probability sample of PyPI. The frozen GO rule required at least 100 projects with three reconstructable attested releases, reconstruction of at least 85% of usable GitHub-attested records, security-relevant deltas in at least 30 projects across at least three categories, mutation recall of at least 0.90, mutation precision of at least 0.95, at least 50% alert reduction relative to raw text diff, and at least three public case replays. We began with 250 projects and extended in deterministic blocks of 50. The binding evidence-volume count was 78 at 250 projects and 89 at 300, both below the threshold of 100; it reached 105 at 350, when all seven gates first passed. No project was selected or removed based on observed semantic deltas.

For each project, releases were ordered by earliest file-upload time. We kept at most the ten most recent non-yanked final PEP 440 releases and selected an sdist when available, otherwise the lexicographically first file. The primary pairing is adjacent among reconstructable GitHub-attested records. Because an unprovenanced or non-GitHub

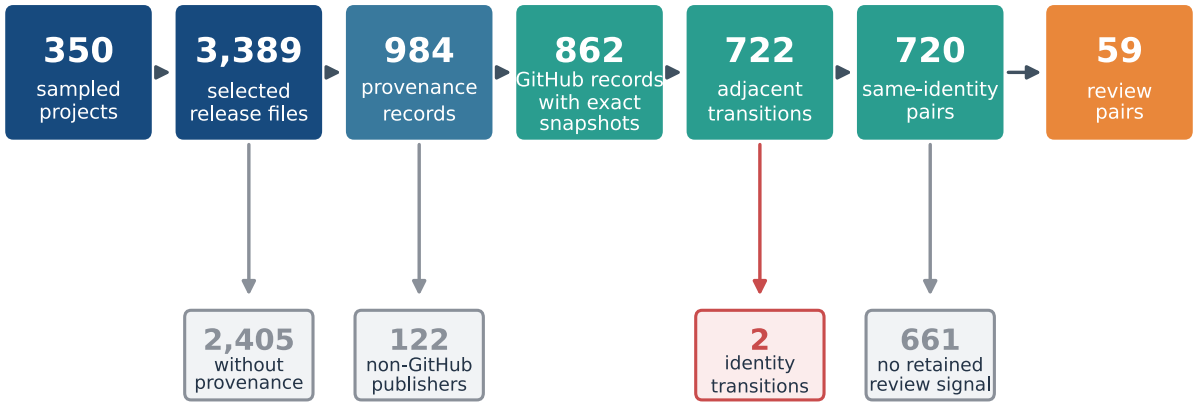


Fig. 2. Measurement flow and analysis branches. *Note.* Counts change units across stages. Branches indicate scope exclusions and analysis routing.

selected release may be skipped, we separately evaluate strict adjacency inside the frozen ten-release window.

The analysis uses four non-interchangeable units:

1. 720 same-identity release pairs, the denominator for queue rates;
2. 59 retained candidate pairs, the human-review unit;
3. 81 category instances on those pairs, because a pair can carry several queue categories; and
4. 275 atomic deltas across all 720 pairs, of which 196 occur inside the 59 retained pairs.

Counts and percentages are always labeled with their unit. Project-clustered 95% intervals use 2000 bootstrap resamples of projects. Cross-file proportions use Wilson intervals. We report effect sizes and intervals, not post-hoc null-hypothesis tests.

4.2. Operational baselines

All temporal baselines use the same 720 same-identity pairs:

- **Raw full-workflow diff** flags unequal workflow bytes.
- **Normalized full-workflow AST diff** removes formatting and key order but compares the entire parsed workflow.
- **Publish-slice structural diff** compares normalized syntax only on the publishing slice. It controls for AttestDiff's path restriction without applying directional semantics.
- **Single-snapshot rule scan** flags whether the newer snapshot contains any state matching the same security vocabulary.
- **Rule-output delta** fingerprints findings produced by AttestDiff's frozen single-snapshot rule vocabulary on both sides and flags a changed finding set. It is not the output of an external commercial or research scanner. This closest rule-based temporal baseline removes persistent findings but does not compare old/new authority using the guarded directional comparison rules.
- **Zizmor temporal finding delta** runs the independent Zizmor 1.29.0 scanner (Woodruff, 2026) on both exact workflow snapshots. The frozen primary configuration uses the auditor persona, severity at least low, confidence at least medium, offline operation, and no repository config. Finding identities retain the audit identifier and symbolic YAML route but exclude filenames and line offsets. A pair enters this queue when the newer snapshot adds at least one finding; the regular persona and any-output-change definitions are reported as sensitivities.

An identity-transition monitor is evaluated separately over all 722 reconstructed transitions because those two transitions are intentionally outside AttestDiff's semantic branch. Queue rate and overlap measure workload and distinction; without human judgments they are not accuracy metrics.

4.3. Conformance, sensitivity, and external audit

We froze 20 authored workflow-pair transformations covering permissions, triggers, filters, OIDC, pinning, job environment declarations, runners, cache/artifact flows, formatting, comments, key order, and job renaming. Thirteen cases were designed to enter the candidate queue, seven were controls, and the positive cases contain 15 expected semantic labels. These cases test implementation conformance, not deployment accuracy.

Two empirical sensitivities address sampling assumptions. First, strict adjacency keeps only pairs consecutive in the complete selected-release stream inside the frozen ten-final-release window. Second, the primary collection contained 984 selected representative-file provenance records. Re-querying the corresponding release file lists showed that all 984 releases also had a second provenance-advertised file, yielding 984 eligible within-release file pairs. From this conditioned frame we draw a deterministic simple random sample without replacement of 200 (seed 20260727), preferring a wheel as the comparison file. We compare publisher identity and workflow SHA across files; this tests the representative-file simplification but does not estimate all PyPI releases.

The frozen stratum-blinded external audit uses two reviewers who had no prior involvement in AttestDiff, no prior collaboration or institutional relationship with the authors, and no declared conflict of interest. ER1 was a third-year software-engineering master's student with two years of relevant review and vulnerability-analysis experience; ER2 was a first-year cyberspace-security doctoral student with three years of relevant experience. Each received all 59 candidates plus the same seeded 59-pair simple random sample from the 290 eligible raw-diff/non-AttestDiff controls. They assigned exactly one of four judgments: *warrants security review*, *benign expected maintenance*, *insufficient public context*, or *cannot determine*, and recorded time and rationale. Review-worthy meant that public evidence reasonably warranted maintainer or security confirmation of an authority, dependency-integrity, or trust-boundary change. Benign maintenance was sufficiently explained without further security review; context-limited cases depended on unavailable policy or private state; cannot-determine was reserved for malformed, inaccessible, inconsistent, or out-of-codebook evidence. Illustrative codebook cases were a new explicit write permission, a formatting-only refactor, removal of a job environment declaration whose protection was private, and an inaccessible or malformed diff, respectively. The reviewers saw immutable workflow URLs and unified diffs but not AttestDiff's decision, semantic labels, or selection stratum. The coordinator hashed both first-round sheets before unblinding and validated every field against the frozen codebook. The reviewers then independently reconsidered only their 15 disagreements after a common clarification, leaving seven for joint adjudication; both confirmed the final adjudication sheet. First-round

agreement remains the primary reliability estimate, while reconsideration is a secondary sensitivity result. The earlier author audit of the same rows is retained only as a construct-sensitivity comparison. Sampling weights do not extend either audit to the 371 raw-text-unchanged pairs or to PyPI as a whole.

5. Results

5.1. RQ1: Exact release-bound reconstruction is feasible

Fig. 2 reports the evidence flow. The 350 projects yielded 3389 selected release files. PyPI exposed provenance for 984 files: 862 identified a GitHub publisher and 122 a non-GitHub publisher kind. All 862 in-scope records produced the referenced public workflow snapshot at the attested commit. The 122 non-GitHub records are scope exclusions, not GitHub reconstruction failures. At least one snapshot was reconstructed for 140 projects and at least three for 105.

The reconstructed streams contained 722 adjacent transitions. Two changed publisher identity and entered the separate identity branch. The remaining 720 same-identity pairs came from 122 projects and form the semantic-analysis denominator. This conditioned denominator demonstrates feasibility; it does not estimate PyPI-wide provenance adoption or identity stability.

5.2. RQ2: Stable identities contain mixed-direction change

AttestDiff retained 59/720 pairs (8.2%; project-clustered 95% CI: 6.0%–10.6%) from 41 projects. Those pairs contained 81 queue-category instances. The largest category was a newly introduced mutable dependency reference on the publishing slice (23 pair instances), followed by newly explicit OIDC exposure (17), changed dynamic authority expressions (14), and changed local reusable workflows (9). Event expansion, artifact download, and permission expansion each occurred in five pairs; cache restore occurred in two and literal filter broadening in one.

The candidate-category distribution and atomic direction ledger answer different questions. Across all 720 pairs, the tool recorded 275 atomic deltas: 29 expansions, 152 restrictions, 76 review signals, and 18 unknowns. Within the 59 retained pairs, there were 196 atomic deltas: 29 expansions, 74 restrictions, 76 review signals, and 17 unknowns. Thus, a candidate can contain hardening changes and uncertainty alongside a review-triggering expansion. The tool preserves this mixed evidence instead of equating a changed workflow with degradation.

5.3. RQ3: Directional differencing yields a distinct queue

Table 3 compares the operational queues, while Fig. 3 highlights the four most informative workload contrasts. Raw and normalized full-workflow differencing flagged 349/720 (48.5%) and 347/720 (48.2%) pairs. Restricting comparison to the publishing slice reduced the structural queue to 294/720 (40.8%), showing that path selection alone does not explain AttestDiff's 59/720. A single-snapshot scan marked 678/720 (94.2%) because persistent states are repeated.

The independent Zizmor temporal baseline marked 236/720 pairs (32.8%; project-clustered 95% CI: 27.4%–38.4%). It overlapped AttestDiff on 48 pairs, with 188 Zizmor-only and 11 AttestDiff-only pairs. The regular-persona sensitivity marked 190/720 (26.4%) and overlapped AttestDiff on 39. Allowing either added or removed findings increased the auditor-persona queue to 259/720 (36.0%). Thus, differencing an established scanner removes persistent findings but still yields a substantially broader baseline queue.

The internal rule-output-delta baseline marked 129/720 (17.9%). Its overlap with AttestDiff was 38 pairs: 91 were baseline-only and 21 were AttestDiff-only. Therefore, AttestDiff is neither merely an 83.1% reduction of raw diffs nor a nested filter over scanner changes. The divergence comes from comparing release-bound old/new values, applying guarded direction, and retaining incomplete-context changes that a scanner delta may represent differently. Within the frozen 118-row external-audit sample, the Zizmor auditor queue contained all 33 final review-worthy judgments, but also 50 benign-maintenance and two context-limited judgments. This is a descriptive comparison within a stratified sample, not a precision or recall estimate for Zizmor.

All 20 authored conformance cases passed. Full-text comparison matched all 13 positive transformations but rejected none of the seven formatting/structure controls; normalized AST rejected two. AttestDiff matched all 13 positive cases, rejected all seven controls, and matched all 15 expected semantic labels. The terms true positive, false positive, precision, and recall are intentionally avoided here because the cases were authored against the specification rather than sampled from deployment. The complete conformance table remains in the artifact.

5.4. RQ4: Findings survive sensitivity and human review

The strict-adjacency subset contained 718/720 same-identity pairs from 121 projects. It retained all 59 AttestDiff candidates and every category count. Raw, AST, publish-slice, snapshot, and rule-delta baselines changed by only one or two pairs. Hence skipped selected releases do not drive the retained result inside the frozen recent-release window. This does not prove consecutiveness outside that ten-release window; the complete counts remain in the artifact.

The cross-file funnel was 984 selected representative provenance records, 984 releases eligible for a second advertised file, 200 seeded SRS releases, and 175 evaluable GitHub/GitHub file pairs. The other 25 sampled releases had non-GitHub status for both files. Among the 175 evaluable pairs, publisher identity matched in 175/175 (100%; Wilson 95% CI: 97.85%–100%) and workflow SHA matched in 174/175 (99.43%; 96.83%–99.90%). The one disagreement was `PyNaCl` 1.6.2: the sdist and sampled wheel named the same identity and path but different workflow SHAs. The representative-file policy is therefore highly consistent in this deterministic sample, but not universally interchangeable. A full study should either compare every attested file or retain a within-release disagreement branch.

Fig. 4 shows that candidate pairs appear in every frozen popularity block. Rates range from 3.3% to 12.1%, with wide, project-clustered intervals in smaller blocks. These are descriptive sensitivity results, not evidence of a popularity effect.

Stratum-blinded external audit and agreement. The primary first-round four-class agreement was 87.3%, with Cohen's $\kappa = 0.743$ and 15 disagreements. Collapsing the outcome to review-worthy versus other gave 88.1% agreement and $\kappa = 0.716$. Targeted independent reconsideration increased four-class agreement to 94.1% ($\kappa = 0.877$), leaving seven jointly adjudicated rows. Because the second round targeted known disagreements, these higher values are sensitivity results and do not replace the first-round reliability estimates.

In the final candidate census, 33/59 transitions (55.9%; Wilson 95% CI: 43.3%–67.8%) warranted security review, 21/59 (35.6%) were benign expected maintenance, and 5/59 (8.5%) required unavailable public context. All 59 sampled non-alerts were judged benign; the Wilson upper 95% bound for their review-worthy rate is 6.1%. The candidate-minus-control difference was 55.9 percentage points, with a project-clustered bootstrap 95% interval of 41.8–69.7 percentage points across 20,000 resamples of 70 projects (see Table 4).

Category instances overlap when one pair contains several changes, so Table 5 is not a partition of 59 pairs. New unpinned dependencies produced 20/23 review-worthy judgments; event expansion produced 5/5, and permission expansion and new artifact download each produced 4/5. OIDC expansion was review-worthy in 6/17 instances, while changed local reusable workflows yielded four review-worthy and five context-limited judgments. The latter result supports retaining unresolved reusable-workflow context rather than forcing a binary decision.

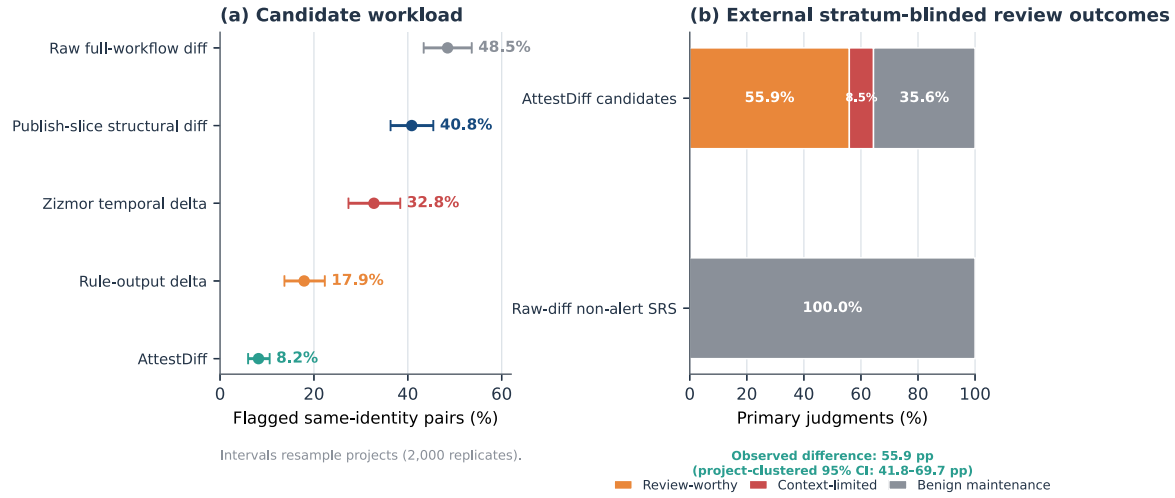


Fig. 3. Candidate workload and external-review outcomes.
Note. Panel (a) reports project-clustered 95% intervals. Panel (b) reports final four-class consensus judgments from the stratum-blinded external audit within each 59-pair stratum; no item received the fourth *cannot determine* judgment.

Table 3
Operational baselines for 720 same-identity pairs.

Method	Flagged	Queue rate	Clustered 95% CI	Overlap	Baseline-only	AttestDiff-only
Raw full-workflow diff	349	48.5%	43.4%–53.6%	59	290	0
Normalized full-workflow AST diff	347	48.2%	43.1%–53.3%	59	288	0
Publish-slice structural diff	294	40.8%	36.3%–45.4%	59	235	0
Single-snapshot rule scan	678	94.2%	88.8%–98.3%	59	619	0
Zizmor temporal finding delta	236	32.8%	27.4%–38.4%	48	188	11
Internal rule-output delta	129	17.9%	13.7%–22.3%	38	91	21
AttestDiff	59	8.2%	6.0%–10.6%	59	0	0

Note."Overlap" is the number also retained by AttestDiff. Intervals resample projects. The Zizmor row uses the frozen auditor-persona added-finding queue; none of these columns is a precision estimate.

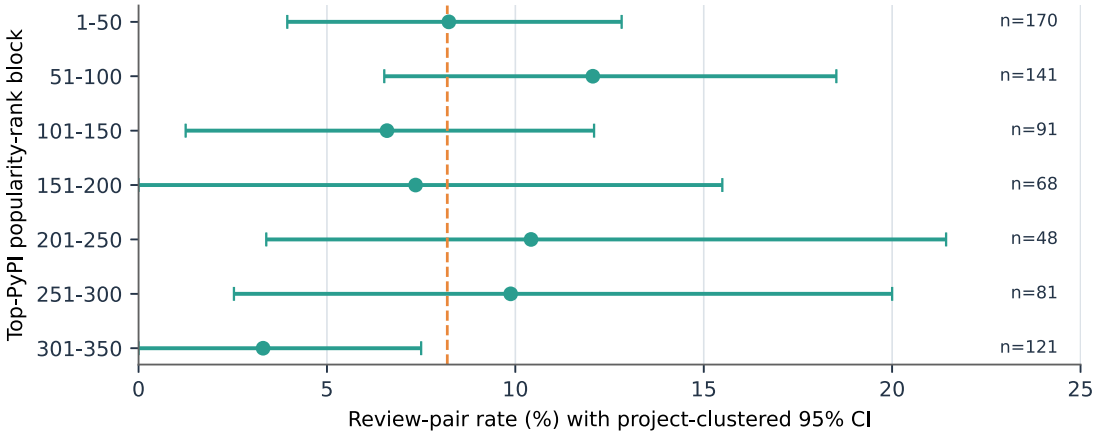


Fig. 4. Candidate rate by popularity-rank block.
Note. Points are deterministic block rates, horizontal intervals are project-clustered 95% intervals, and labels show the available pair count. Blocks are descriptive strata, not independent probability samples.

Table 4
Final stratum-blinded external-audit judgments.

Audit stratum	*n*	Review-worthy	Benign maintenance	Context-limited	Cannot determine
AttestDiff candidates (census)	59	33 (55.9%)	21 (35.6%)	5 (8.5%)	0
Raw-diff non-alerts (SRS)	59	0 (0.0%)	59 (100%)	0 (0.0%)	0

Note. The candidate row is a census; the non-alert row is a seeded simple random sample from 290 raw-diff controls. Percentages are within row, no item was assigned *cannot determine*, and "review-worthy" is not a vulnerability label.

Table 5
External-audit outcomes by candidate category.

Candidate category	Instances	Review-worthy	Benign	Context-limited	Review-worthy yield
New unpinned dependency	23	20	3	0	87.0%
OIDC expansion	17	6	11	0	35.3%
Dynamic expression change	14	7	7	0	50.0%
Local reusable-workflow change	9	4	0	5	44.4%
Event expansion	5	5	0	0	100%
New artifact download	5	4	1	0	80.0%
Permission expansion	5	4	1	0	80.0%
New cache restore	2	1	1	0	50.0%
Filter broadening	1	1	0	0	100%

Note. The unit is a category instance, not a unique pair, and categories may overlap. Review-worthy yield equals Review-worthy/Instances within each category: the share whose pair received the final consensus judgment *warrants security review*. It is a category-instance descriptive rate, not pair-level precision.

Table 6
Review-card exemplar for `python-dotenv` 1.2.1→1.2.2.

Field	Predecessor (1.2.1)	Current (1.2.2)	Decision	Evidence retained
Publisher identity	GitHub issuer, repository, workflow path, and publisher environment	Same four-field tuple	Same-identity comparison	Attestation records and exact workflow SHAs
`contents` permission	None under complete explicit context	`write`	Observable expansion	`release.yml`, current line 16
Action reference	Absent	`actions-gh-pages@v4`	Review signal	`release.yml`, current line 44
Interpretation	Maintainer-described GitHub Pages permission fix		Review, not vulnerability	Exact SHAs, URLs, source lines, and content hashes

Note. The artifact retains the full public URLs and content hashes; the table abbreviates them for print.

The earlier internal author audit is retained only as a sensitivity analysis. Its final-label agreement with the external consensus was 92.4% ($\kappa = 0.846$) for four classes and 94.1% ($\kappa = 0.862$) after binary collapse. The authors had marked four sampled controls review-worthy, whereas the external consensus judged all four benign. Their non-selection maps to specific components. *filelock:* removing `workflow_dispatch` and `pull-requests: read` emitted only restrictions, which the directional candidate rule does not retain; a separately deleted release-tagging job also lay outside (P(W)). *Flask:* the removed SLSA-provenance job was in the predecessor's publishing ancestry, but provenance-job removal is absent from the frozen six-family semantic taxonomy, so no eligible delta was emitted. *Click:* the same taxonomy boundary applied to its removed SLSA-provenance job. *NumPy:* a new nightly-upload job lay outside (P(W)), while artifact upload/download already existed on the release publishing slice in both snapshots; the remaining SHA-pinned action updates and build refactoring emitted neither a new-handoff signal nor an authority-expansion category. These are case-specific construct boundaries, not externally confirmed misses. Conversely, zero review-worthy controls in a 59-row sample cannot establish complete recall, especially for the 371 raw-text-unchanged pairs that were not sampled.

Both strata were reviewed through the same URL-and-unified-diff interface. We did not compare review cards against conventional diffs under randomized or counterbalanced conditions, so we make no claim that the cards improve review speed or judgment quality.

5.5. Review cards preserve actionable public evidence

Table 6 gives a real card for `python-dotenv` 1.2.1→1.2.2. The four publisher-identity fields remain unchanged, while the new attested workflow grants `contents: write` under complete explicit context and introduces `peaceiris/actions-gh-pages@v4`. The card points to the exact current workflow lines and retains both workflow SHAs. The PyPI publishing sink (line 35) and the Pages action (line 44) occur in the same `publish` job, so both lie in the sink-containing publishing slice. The maintainer described this transition as a GitHub Pages permission fix; it is evidence that the card captures a reviewable change, not evidence of a vulnerability.

Two additional maintainer-described transitions replayed the frozen taxonomy. In `trove-classifiers`, isolating OIDC to the publish job produced an explicit-permission change, artifact handoff, and newly introduced mutable dependency reference while retaining the hardening context. In `tzdata`, build/publish isolation produced trigger, permission, artifact, local-workflow, and dynamic-expression deltas. These purposive replays show evidence feasibility and mixed direction; they do not estimate prevalence or correctness.

The cache-populating acquisition transferred 418.6 MB through 2427 network requests in 86.9 s on the study machine. That pass preceded a correction that reclassified 122 Google-publisher records from decode errors to non-GitHub scope exclusions. The corrected analysis replayed from cache in 14.2 s. These are cold/warm observations, not a controlled benchmark.

6. Discussion

6.1. What the attestation adds

The result is not that attestations are ineffective. The attestation supplies the identity, release association, and exact commit that make a defensible comparison possible. AttestDiff adds a downstream question: whether the public authority behind that identity changed relative to its attested predecessor. This is consistent with the provenance claim boundary and with empirical work showing that signing and provenance require usable interpretation, not only cryptographic availability (Schorlemmer et al., 2024a; Kalu et al., 2026).

The proposed control stack is complementary. An identity-transition method handles the two off-ramp cases. A snapshot scanner looks for persistent injection or policy weaknesses. A repository-history analysis supports maintenance studies. AttestDiff prioritizes newly observable, publishing-relevant semantics in exact attested release pairs. The independent Zizmor comparison prevents an inflated novelty claim: temporal differencing of an established scanner already removes persistent findings, but its 236-pair queue and 48-pair overlap with AttestDiff demonstrate a broader and only partly overlapping review unit.

6.2. Operational use and decision limits

An index, organization, or maintainer tool could attach the review card to a new attested release. Identity change would invoke release-authority review. Stable identity would invoke AttestDiff; a card would show the predecessor, direction, confidence, old/new values, and source locations. Restrictions could explain or downgrade a mixed candidate; expansions and incomplete signals could request confirmation. The tool should not auto-block releases based on the current 8.2% queue rate.

The external audit supplies decision evidence without turning review judgments into vulnerability labels. The 55.9% candidate yield versus 0/59 in sampled controls supports AttestDiff as a prioritization instrument, while the five context-limited candidates validate the decision not to force unavailable policy facts into a binary label. The result also sets a practical limit: 21 candidates were benign maintenance, and the control interval still permits a nonzero miss rate. Accordingly, AttestDiff can enrich the candidate set presented to reviewers but should neither auto-block a release nor be described as a complete detector.

6.3. Triage and benign-change mitigation

The category-level audit suggests an operational order without changing the frozen detector. Confirmed event, filter, permission, dependency-pinning, and artifact-handoff changes should appear before context-dependent signals. A card should merge overlapping categories for one pair, show co-occurring restrictions, and demote a restriction-only pair when observable context is complete. Dynamic expressions, OIDC changes, and local reusable-workflow changes should request the missing effective value or retrieve the referenced public reusable workflow before escalation. Project-specific approvals can suppress an identical, previously reviewed migration, but the old/new hashes and rationale should remain in the audit trail.

For example, the `python-dotenv` card in Table 6 becomes one item: show the confirmed `contents` expansion first, retain the new mutable action reference as context-dependent, and check both against the recorded deployment rationale. Approval-based suppression matches only the exact old/new workflow hashes; changed hashes return to review.

These mitigations are workflow recommendations, not post-hoc detector tuning. In particular, the 59 audited candidates are too small to justify deleting a low-yield category. Any suppression policy derived from these outcomes should be frozen and evaluated on a new release cohort before deployment.

7. Threats to validity

External validity. The dated download ranking is a convenience frame of 350 popular projects, not a probability sample. We cover at most ten recent final releases, one representative file per primary release, GitHub publishers, and public workflows. Results do not estimate PyPI-wide prevalence, older histories, less-popular projects, private workflows, or other ecosystems. The 200-release cross-file sample reduces uncertainty about the representative file but found one real SHA disagreement and cannot justify universal equivalence. Candidate rates in every frozen 50-project popularity block show that the phenomenon is not confined to one block, but the 3.3%–12.1% range is descriptive and does not remove popularity bias.

Internal validity. Primary pairs can skip an unprovenanced or non-GitHub selected release. The strict-consecutive sensitivity preserves all 59 candidates, but only within the frozen ten-release window. Release streams can also interleave maintenance branches. Parser, sink-detection, and path-projection errors remain possible despite conformance tests. Results depend on PyPI- and GitHub-served public evidence at collection time.

Construct validity. An observable expansion is not necessarily exploitable; a review signal is not a vulnerability; and an unknown is not evidence of safety. Repository defaults, job environment protection, branch policy, private dependencies, matrices, and expressions can change runtime authority. Baseline queue rates, 20 authored cases, and three purposive public replays cannot establish vulnerability prevalence. The two-reviewer audit measures an operational review-worthiness construct, not exploitability. The reviewers were external to the project and had relevant graduate-level software/security experience, but two reviewers cannot establish consensus across a broader expert population. First-round agreement and disagreement counts are therefore reported before clarification or adjudication.

Statistical conclusion validity. Pairs within a project are dependent, so intervals resample projects. Rank blocks are deterministic sensitivity strata rather than independent random samples. The cross-file sensitivity uses a seeded sample from 984 eligible selected releases and Wilson intervals; it does not estimate releases outside that conditioned frame. No causal claim is made from release intervals, popularity rank, or semantic change.

The control estimate comes from 59 of 290 eligible raw-diff non-alerts. Its zero observed review-worthy count has a 6.1% Wilson upper bound and remains conditioned on that frame. The project-clustered enrichment interval resamples 70 projects but does not turn the stratified audit into a randomized treatment comparison. We therefore report candidate yield and control uncertainty rather than precision or recall over all 720 pairs.

Provenance validity. The implementation hashes and decodes PyPI-served provenance but does not re-run full cryptographic verification of every attestation. The supported claim is exact reconstruction from public served evidence, not a replacement verifier or proof that the named workflow was uncompromised.

8. Ethics and reproducibility

The study uses read-only requests to public PyPI metadata, public attestations, and public GitHub source. It does not install packages, execute workflows, mint credentials, assume cloud roles, probe private infrastructure, or contact maintainers. Responses are cached and content-hashed. The anonymous Supplementary ZIP contains the frozen protocols and configuration, dated sampling-frame snapshot, environment, public project/release inventory, source and run manifests, reconstruction records, a de-identified 720-pair analytical ledger, the 20 conformance cases, independent-scanner protocol and aggregate outputs, bootstrap and sensitivity materials, public replays, figure-generation scripts, external-audit codebook and sampling procedure, completed-sheet hashes, agreement statistics, final aggregate output, and 118 de-identified row-level consensus records.

The row-level audit table removes direct project names, versions, workflow URLs and digests, original audit identifiers, and local paths, and uses opaque HMAC-derived identifiers. Raw completed sheets, reviewer declarations, coordinator keys, private salts, and row-to-project linkage remain restricted locally. This is direct-identifier removal rather than a formal guarantee against inference from distinctive public workflow semantics. The frozen validation rejects incomplete or out-of-vocabulary judgments. Any public disclosure or maintainer contact requires separate authorization and responsible review.

9. Conclusion

AttestDiff contributes a predecessor-aware, release-bound semantic comparison unit for the blind spot between identity-level release-authority measurement and snapshot or repository-wide workflow scanning. It begins where publisher-identity comparison stops: after an adjacent release pair retains the same four-field attested identity. It then reconstructs the exact workflows at the attested commits, isolates each transitive publishing slice, and applies a set of guarded directional

comparison rules. The resulting review card preserves expansions, restrictions, incomplete-context review signals, unknowns, and their source evidence rather than reducing a transition to "changed/not changed" or a vulnerability verdict. This combination of comparison unit, guarded semantics, and evidence retention is the paper's principal methodological contribution.

The frozen study demonstrates that the comparison is reconstructable and selective at useful scale. Across 350 popular PyPI projects, 862 exact workflow snapshots yielded 720 same-identity adjacent pairs. AttestDiff retained 59 directional candidates, compared with 236 added-finding deltas from independent Zizmor, 129 internal rule-output deltas, and 349 raw workflow changes. Strict adjacency retained all 59 candidates, while the cross-file analysis exposed one genuine limitation of representative-file selection. In the external audit, first-round four-class agreement was 87.3% ($\kappa = 0.743$); final consensus judged 33/59 candidates review-worthy and five context-limited, compared with 0/59 review-worthy sampled non-alerts. The 55.9-point enrichment had a project-clustered 95% interval of 41.8–69.7 points. Category-specific outcomes further show where confirmed expansions and
context-dependent signals require different triage.

The findings do not establish vulnerability, exploitability, or PyPI-wide prevalence. AttestDiff cannot observe private policy, runtime-resolved authority, or a compromise that leaves both publisher identity and public workflow semantics unchanged. Its practical role is therefore to focus release review, not to auto-block publication. The external audit does not establish complete recall, analyst-efficiency improvement, or consensus beyond two reviewers. Independent replication, a less popularity-biased cohort, broader ecosystem coverage, and comparisons across every provenance-advertised file would strengthen the evidence. The central conclusion is that stable publisher identity is a necessary release-path anchor but is not sufficient evidence of stable publishing authority.

CRediT authorship contribution statement

Qingshui Xue: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Pandong Xue:** Writing – review & editing, Writing – original draft, Validation, Investigation. **Zhimin Wang:** Supervision. **Haifeng Ma:** Writing – review & editing, Funding acquisition.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used OpenAI ChatGPT to improve language and readability. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the article.

Funding

This research was supported by the Planning Project of Shanghai Social Science (Grant No. 2022BGL003).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.cose.2026.105209.

Data availability

This study uses public PyPI metadata, public PyPI attestations, and public GitHub source. An anonymous Supplementary ZIP accompanies the submission. It contains the implementation and environment, frozen protocols and stopping rules, dated ranking snapshot, public project/release inventory, content-hashed manifests, de-identified 720-pair analytical ledger, all 20 conformance cases, independent-scanner protocol and aggregate outputs, bootstrap and sensitivity scripts, external-audit codebook and sampling procedure, hashes of both first-round review sheets, agreement and adjudication summaries, 118 de-identified final consensus records, and table/figure regeneration scripts. The supplement is not publicly archived and has no public URL, but it permits verification of the primary aggregate claims without contacting the authors. Raw completed review sheets and the restricted row-to-project linkage are not distributed; subject to third-party terms and responsible-disclosure review, additional restricted material may be requested from the corresponding author.

References

Ayala, J., Tung, Y.-J., Garcia, J., 2025. A mixed-methods study of open-source software maintainers on vulnerability management and platform security features. In: 34th USENIX Security Symposium. USENIX Security 25, USENIX Association, pp. 2111–2130, URL https://www.usenix.org/conference/usenixsecurity25/presentation/ayala.

Chaiwut, N., Nikiforakis, N., 2025. Time for actions: A longitudinal study of the GitHub Actions marketplace. In: 2025 IEEE Secure Development Conference. SecDev, pp. 118–128. http://dx.doi.org/10.1109/SecDev66745.2025.00023.

Delicheh, H.O., Decan, A., Mens, T., 2024. Quantifying security issues in reusable JavaScript actions in GitHub workflows. In: 21st International Conference on Mining Software Repositories. MSR, ACM, http://dx.doi.org/10.1145/3643991.3644899.

Delicheh, H.O., Mens, T., 2024. Mitigating security issues in GitHub Actions. In: 2024 ACM/IEEE Workshops on Engineering and Cybersecurity of Critical Systems and Software Vulnerability. ACM, http://dx.doi.org/10.1145/3643662.3643961.

Fares, M., Gamage, Y., Baudry, B., 2026. Unpacking security scanners for GitHub Actions workflows. http://dx.doi.org/10.48550/arXiv.2601.14455, arXiv preprint arXiv:2601.14455.

GitHub, 2024. Artifact attestations is generally available. URL https://github.blog/changelog/2024-06-25-artifact-attestations-is-generally-available/. (Accessed 27 July 2026). GitHub Changelog.

GitHub, 2026a. OpenID connect reference. URL https://docs.github.com/en/actions/reference/security/oidc. (Accessed 27 July 2026). GitHub documentation.

GitHub, 2026b. Secure use reference for GitHub Actions. URL https://docs.github.com/en/actions/reference/security/secure-use. (Accessed 27 July 2026). GitHub documentation.

GitHub, 2026c. Workflow syntax for GitHub Actions. URL https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax. (Accessed 27 July 2026). GitHub documentation.

Gu, Y., Ying, L., Chai, H., Pu, Y., Duan, H., Gao, X., 2024. More haste, less speed: Cache related security threats in continuous integration services. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1179–1197. http://dx.doi.org/10.1109/SP54263.2024.00138.

Gu, Y., Ying, L., Chai, H., Qiao, C., Duan, H., Gao, X., 2023a. Continuous intrusion: Characterizing the security of continuous integration services. In: 2023 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1561–1577. http://dx.doi.org/10.1109/SP46215.2023.10179471.

Gu, Y., Ying, L., Pu, Y., Hu, X., Chai, H., Wang, R., Gao, X., Duan, H., 2023b. Investigating package related security threats in software registries. In: 2023 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1578–1595. http://dx.doi.org/10.1109/SP46215.2023.10179332.

He, H., Vasilescu, B., Kästner, C., 2025. Pinning is futile: You need more than local dependency versioning to defend against supply chain attacks. Proc. ACM Softw. Eng. 2 (FSE), http://dx.doi.org/10.1145/3715728.

Ingram, D., 2023. Introducing trusted publishers. URL https://blog.pypi.org/posts/2023-04-20-introducing-trusted-publishers/. Python Package Index Blog.

Ingram, D., 2024. PyPI now supports digital attestations. URL https://blog.pypi.org/posts/2024-11-14-pypi-now-supports-digital-attestations/. (Accessed 27 July 2026). Python Package Index Blog.

Kalu, K.G., Okorafor, S., Singla, T., Chen, S., Torres-Arias, S., Davis, J.C., 2026. Why johnny adopts identity-based software signing: A usability case study of Sigstore. In: 35th USENIX Security Symposium. USENIX Security 26, USENIX Association, URL https://www.usenix.org/conference/usenixsecurity26/presentation/kalu.

Kalu, K.G., Singla, T., Okafor, C., Torres-Arias, S., Davis, J.C., 2025. An industry interview study of software signing for supply chain security. In: 34th USENIX Security Symposium. USENIX Security 25, USENIX Association, pp. 81–100, URL https://www.usenix.org/conference/usenixsecurity25/presentation/kalu.

Koishybayev, I., Nahapetyan, A., Zachariah, R., Muralee, S., Reaves, B., Kapravelos, A., Machiry, A., 2022. Characterizing the security of GitHub CI workflows. In: 31st USENIX Security Symposium. USENIX Security 22, USENIX Association, pp. 2747–2763, URL https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev.

Kubo, Y., Kanei, F., Akiyama, M., Wakai, T., Mori, T., 2026. Action required: A mixed-methods study of security practices in GitHub Actions. In: Network and Distributed System Security Symposium. NDSS, URL https://www.ndss-symposium.org/ndss-paper/action-required-a-mixed-methods-study-of-security-practices-in-github-actions/.

Ladisa, P., Plate, H., Martinez, M., Barais, O., 2023. SoK: Taxonomy of attacks on open-source software supply chains. In: 2023 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1509–1526. http://dx.doi.org/10.1109/SP46215.2023.10179304.

Mazrae, P.R., Decan, A., Mens, T., Wessel, M., 2026. An empirical study of the evolution of GitHub Actions workflows. J. Syst. Softw. 236, 112824. http://dx.doi.org/10.1016/j.jss.2026.112824.

Muralee, S., Koishybayev, I., Nahapetyan, A., Tystahl, G., Reaves, B., Bianchi, A., Enck, W., Kapravelos, A., Machiry, A., 2023. ARGUS: A framework for staged static taint analysis of GitHub workflows and actions. In: 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, pp. 6983–7000, URL https://www.usenix.org/conference/usenixsecurity23/presentation/muralee.

Newman, Z., Meyers, J.S., Torres-Arias, S., 2022. Sigstore: Software signing for everybody. In: 2022 ACM SIGSAC Conference on Computer and Communications Security. CCS, ACM, pp. 2353–2367. http://dx.doi.org/10.1145/3548606.3560596.

Okafor, C.L., Davis, J.C., Torres-Arias, S., 2024. DiVerify: Diversifying identity verification in next-generation software signing. http://dx.doi.org/10.48550/arXiv.2406.15596, arXiv preprint arXiv:2406.15596.

Open Source Security Foundation, 2023. SLSA version 1.0 specification. URL https://slsa.dev/spec/v1.0/. Supply-chain Levels for Software Artifacts.

Python Package Index, 2026a. Integrity API. URL https://docs.pypi.org/api/integrity/. (Accessed 27 July 2026). PyPI documentation.

Python Package Index, 2026b. Security model and considerations for attestations. URL https://docs.pypi.org/attestations/security-model/. (Accessed 27 July 2026). PyPI documentation.

Santos-Grueiro, I., 2026. On good authority: Release-authority measurement for registry-mediated package ecosystems. http://dx.doi.org/10.48550/arXiv.2606.22593, arXiv preprint arXiv:2606.22593.

Saroar, S.G., Nayebi, M., 2023. Developers' perception of GitHub Actions: A survey analysis. In: 27th International Conference on Evaluation and Assessment in Software Engineering. EASE, ACM, pp. 121–130, URL https://arxiv.org/abs/2303.04084.

Schorlemmer, T.R., Burmane, E.H., Kalu, K.G., Torres-Arias, S., Davis, J.C., 2024a. Establishing provenance before coding: Traditional and next-gen software signing. http://dx.doi.org/10.48550/arXiv.2407.03949, arXiv preprint arXiv:2407.03949.

Schorlemmer, T.R., Kalu, K.G., Chigges, L., Ko, K.M., Ishgair, E.A.-M.A., Bagchi, S., Torres-Arias, S., Davis, J.C., 2024b. Signing in four public software package registries: Quantity, quality, and influencing factors. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1160–1178. http://dx.doi.org/10.1109/SP54263.2024.00215.

Torres-Arias, S., Afzali, H., Kuppusamy, T.K., Curtmola, R., Cappos, J., 2019. In-toto: Providing farm-to-table guarantees for bits and bytes. In: 28th USENIX Security Symposium. USENIX Security 19, USENIX Association, pp. 1393–1410, URL https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias.

Valenzuela-Toledo, P., Bergel, A., Kehrer, T., Nierstrasz, O., 2024. The hidden costs of automation: An empirical study on GitHub Actions workflow maintenance. In: 2024 IEEE International Conference on Source Code Analysis and Manipulation. SCAM, IEEE, pp. 213–223, URL https://arxiv.org/abs/2409.02366.

Woodruff, W., 2025. PEP 807: Index Support for Trusted Publishing. Draft Python Enhancement Proposal, Python Software Foundation, URL https://peps.python.org/pep-0807/.

Woodruff, W., 2026. Zizmor: Static analysis for GitHub Actions. URL https://github.com/zizmorcore/zizmor. (Accessed 28 August 2026). Software, version 1.29.0.

Woodruff, W., Tuesca, F., Ingram, D., 2024. PEP 740: Index Support for Digital Attestations. Python Enhancement Proposal, Python Software Foundation, URL https://peps.python.org/pep-0740/.